What is multigrid, and why is it so fast?

optimal solvers for nonlinear problems

Ed Bueler

University of Alaska Fairbanks

Tennessee Tech, October 2026

Outline

1. examples: two elliptic PDE problems
2. simple iterations
3. 2-grid method, and the coarse-grid correction
4. V-cycle multigrid
5. nonlinear multigrid

- I welcome questions at any time!
- they are a *good* use of time

partial differential equations (PDEs)

- today's talk uses the gradient ∇ and the divergence $\nabla\cdot$
  - also the Laplacian, which I will write as ∇^2
- entirely in 2 spatial dimensions
- recall that for points $(x, y) \in \mathbb{R}^2$, functions $u : \mathbb{R}^2 \to \mathbb{R}$, and vector fields $\mathbf{F} = (F_1, F_2) : \mathbb{R}^2 \to \mathbb{R}^2$,

$$\nabla u = \left(\frac{\partial u}{\partial x}, \frac{\partial u}{\partial y}\right) \qquad \textit{slope of } u\textit{, as a vector field}$$

$$\nabla \cdot \mathbf{F} = \frac{\partial F_1}{\partial x} + \frac{\partial F_2}{\partial y} \qquad \textit{positive if } \mathbf{F} \textit{ splays out (diverges)}$$

$$\nabla^2 u = \nabla \cdot (\nabla u) = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \qquad \textit{average concavity of } u$$

- these are all partial derivatives
- if they appear in an equation then it is a partial differential equation

a nonlinear PDE problem

- suppose $\Omega \subset \mathbb{R}^2$ is a domain, a nice open subset

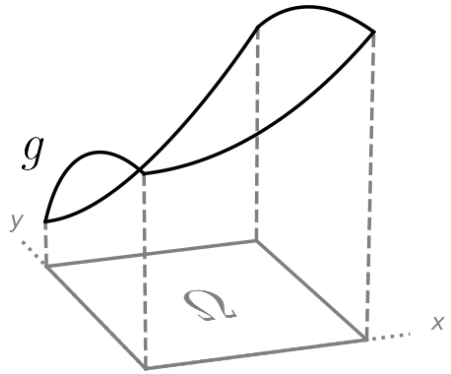
Definition

the area of a surface $z = v(x, y)$ over Ω is the integral

$$A[v] = \int_\Omega \sqrt{1 + |\nabla v|^2}\, dx\, dy$$

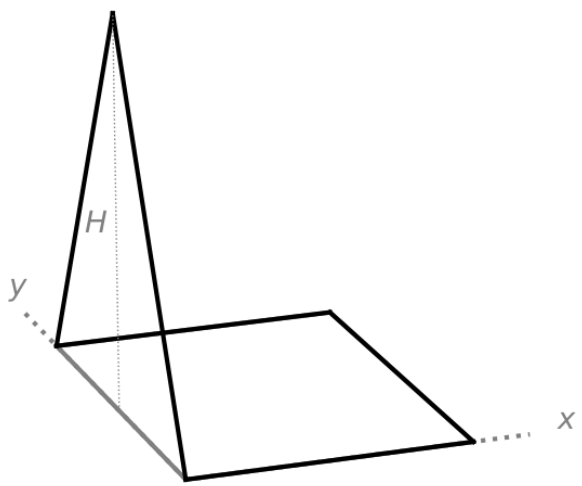
- given a height function g along $\partial\Omega$, a wire frame
- consider the function which minimizes $A[v]$:

$$u \leftarrow \min_{\{v\,:\,v|_{\partial\Omega}=g\}} A[v]$$

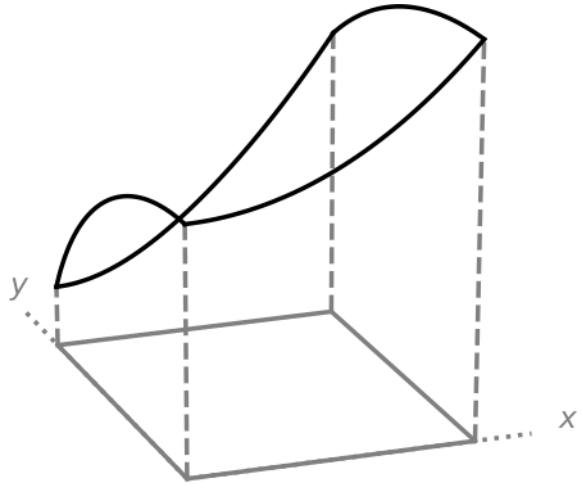


- the minimizer $u(x, y)$ solves the PDE on the next slide

minimal surfaces



tent



catenoid

minimal surface equation

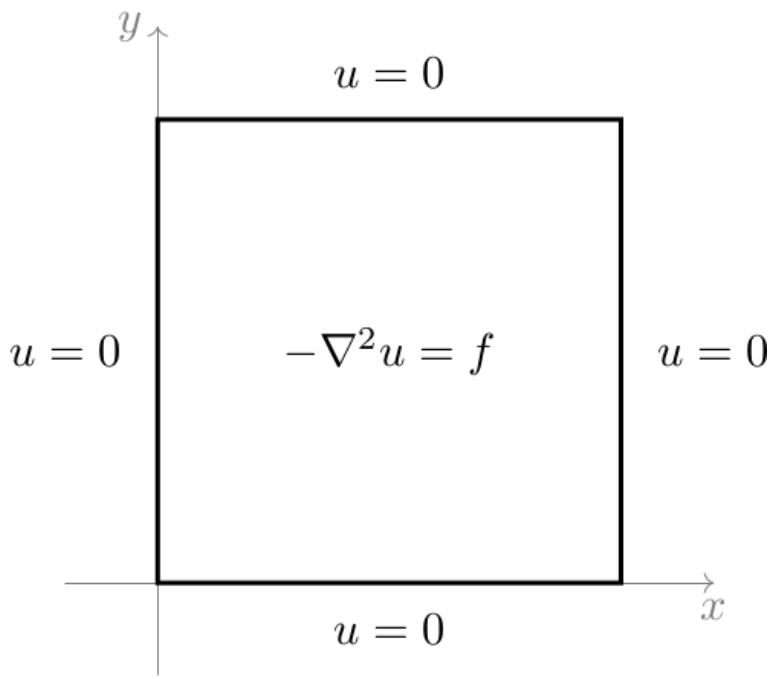
For given boundary function (wire frame) $g(x, y)$, find $u(x, y)$ so that

$$-\nabla \cdot \left(\frac{\nabla u}{\sqrt{1 + |\nabla u|^2}} \right) = 0 \quad \text{on } \Omega, \qquad u\big|_{\partial\Omega} = g.$$

Claim. In the absence of gravity, the surface $z = u(x, y)$ will be the shape of a soap bubble spanning g.

a linear PDE problem

- I will consider only two PDEs today
  1. minimal surface equation *(nonlinear)*
  2. Poisson equation *(linear)*
- both are elliptic PDE boundary-value problems
- problem 2 is easier, so we will start there
- note: Ω is a square in concrete examples



Poisson equation (with homogeneous boundary conditions)

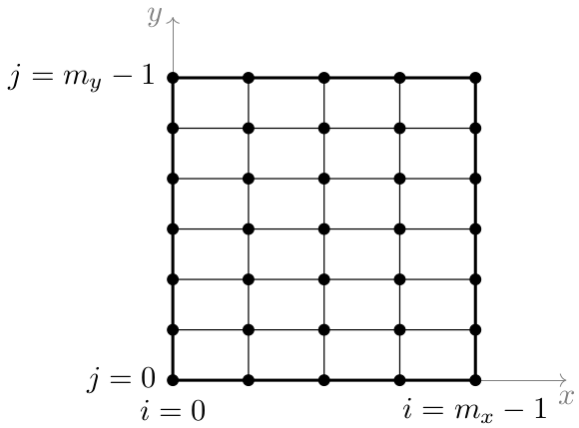
For a given source function $f(x, y)$, find $u(x, y)$ so that

$$-\nabla^2 u = f \quad \text{on } \Omega, \qquad u\big|_{\partial\Omega} = 0.$$

finite difference discretization

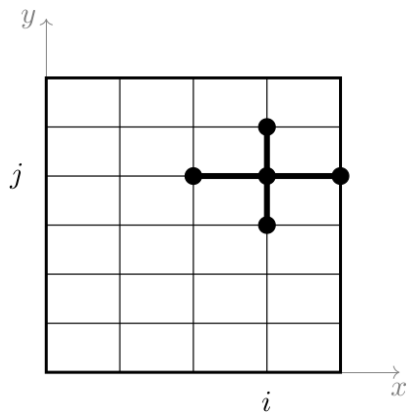
- PDEs are infinite-dimensional problems
- discretization yields a finite system of equations in finitely-many unknowns
- for example, via finite differences (FD):

$$f''(x) = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2} + O(h^2)$$



- assume $\Omega = (0,1)^2$, and put a $m_x \times m_y$ grid on it
- with spacings: $h_x = \frac{1}{m_x - 1}$, $h_y = \frac{1}{m_y - 1}$
- grid points: $(x_i, y_j) = (ih_x, jh_y)$ for $i = 0, \ldots, m_x - 1$ and $j = 0, \ldots, m_y - 1$
- 5-point stencil for the Laplacian:

$$\nabla^2 u(x_i, y_j) \approx \frac{u_{i-1,j} - 2u_{ij} + u_{i+1,j}}{h_x^2} + \frac{u_{i,j-1} - 2u_{ij} + u_{i,j+1}}{h_y^2}$$

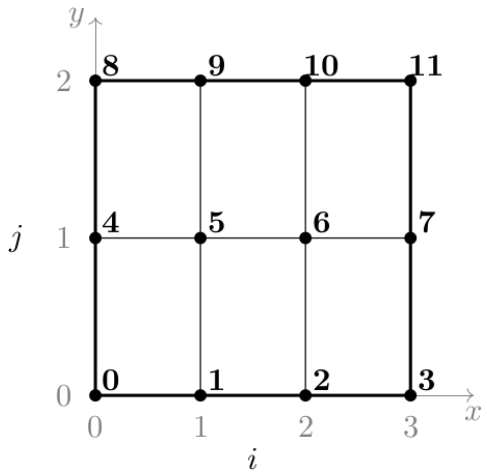


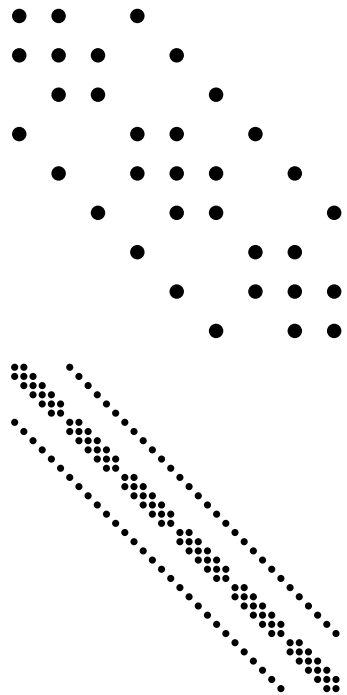
PDE $\overset{\text{discretize}}{\Longrightarrow}$ $A\mathbf{u} = \mathbf{b}$

- discretize $-\nabla^2 u = f$ by FD:
  - $N = m_x m_y$ unknowns on the grid
    - $m_x = 4, m_y = 3$ shown at right
  - unknowns $\{u_{ij}\}$ are globally ordered: $u_\ell = u_{ij}$
- get a linear system for $\mathbf{u} \in \mathbb{R}^N$:

$$A\mathbf{u} = \mathbf{b}$$

  - $A \in \mathbb{R}^{N \times N}$ is sparse
  - A is symmetric positive definite
  - A has $O(1)$ nonzeros per row ← *at most 5*
  - $\mathbf{b} \in \mathbb{R}^N$ has entries $b_\ell = f(x_i, y_j)$
- examples at right:
  - for $m_x = m_y = 5$: 9 non-trivial eqns
  - for $m_x = m_y = 8$: 36 non-trivial eqns





goal: solve $A\mathbf{u} = \mathbf{b}$ in $O(N)$ work

- for our Poisson equation problem, the linear system

$$A\mathbf{u} = \mathbf{b}$$

  has N unknowns
- it can be set-up using $6N$ memory locations
  - A has 5 nonzero entries per row, plus one entry of $\mathbf{b}$

- can we solve $A\mathbf{u} = \mathbf{b}$ in $O(N)$ work and time as $N \to \infty$?
  - if so, the solver is optimal

- let's back up and ask:

  what operations using our A are obviously $O(N)$?

cheap $O(N)$ operations: mat-vec and residual

- we can compute $A\mathbf{v}$ in $O(N)$ flops
  - this is because A has $O(1)$ nonzero entries per row
  - and we do not need to do any work to multiply by zero

Definition

given $\mathbf{v} \in \mathbb{R}^N$, the residual for the linear system $A\,\mathbf{u} = \mathbf{b}$ is

$$\mathbf{r}(\mathbf{v}) = \mathbf{b} - A\mathbf{v}$$

observe that:

$$A\,\mathbf{u} = \mathbf{b} \quad \Longleftrightarrow \quad \mathbf{r}(\mathbf{u}) = 0$$

- evaluating the residual is also an $O(N)$ operation

outline

1. examples: two elliptic PDE problems
2. simple iterations
3. 2-grid method, and the coarse-grid correction
4. V-cycle multigrid
5. nonlinear multigrid

sometimes, simple iterations are cheap $O(N)$ operations

Definition

for the linear system $A\mathbf{u} = \mathbf{b}$ and an invertible matrix $M \in \mathbb{R}^{N\times N}$, simple iteration is

$$\mathbf{v}_{k+1} = \mathbf{v}_k + \alpha\, M^{-1} \underbrace{(\mathbf{b} - A\mathbf{v}_k)}_{=\mathbf{r}(\mathbf{v}_k)}$$

- $\alpha \in \mathbb{R}$ is a tuning parameter, with default $\alpha = 1$
- M is called the preconditioner matrix . . . next slide explains that
- if $M = A$ and $\alpha = 1$ then $\mathbf{v}_{k+1} = \mathbf{u}$, so one iteration solves it!
  - . . . not practical, and usually not $O(N)$
- a simple iteration is $O(N)$ . . . *if* M is the right kind of matrix!

preconditioned linear systems

Definition

if a matrix or linear map $M \in \mathbb{R}^{N \times N}$ is invertible, we say

$$M^{-1} A\mathbf{u} = M^{-1}\mathbf{b}$$

is a (left-) preconditioned system for $A\,\mathbf{u} = \mathbf{b}$

- the preconditioned system has the same solutions as before
- the new residual is $\tilde{\mathbf{r}}(\mathbf{v}) = M^{-1}(\mathbf{b} - A\mathbf{v})$

fast preconditioners?

Definition

a matrix or linear map M is a fast preconditioner if solving $M\mathbf{z} = \mathbf{c}$ requires $O(N)$ work

- example: if A has nonzero diagonal entries, the diagonal $M = D$ is a fast preconditioner
- do *not* actually form the matrix M^{-1} when solving $M\mathbf{z} = \mathbf{c}$
- warning: a preconditioner M can be fast without being effective!
- preconditioning is an apparently simple idea, but in 21st century it is used in all advanced solvers

$O(N)$ simple iterations: Jacobi and Gauss-Seidel

- split A into diagonal and lower/upper triangular parts:

$$A = D + L + U$$

- the linear system can be rearranged using the splitting:

$$\begin{aligned} A\mathbf{u} = \mathbf{b} \quad &\iff \quad D\mathbf{u} = \mathbf{b} - (L+U)\mathbf{u} \\ &\iff \quad \mathbf{u} = \mathbf{u} + D^{-1}(\mathbf{b} - A\mathbf{u}) \end{aligned}$$

- solving $D\mathbf{z} = \mathbf{c}$ is $O(N)$

Definition

a Jacobi iteration applies a fast preconditioner:

$$\mathbf{v}_{k+1} = \mathbf{v}_k + D^{-1}(\mathbf{b} - A\mathbf{v}_k)$$

$O(N)$ simple iterations: Jacobi and Gauss-Seidel

- split A into diagonal and lower/upper triangular parts:

$$A = D + L + U$$

- the linear system can be rearranged using the splitting:

$$\begin{aligned} A\mathbf{u} = \mathbf{b} \quad &\iff \quad (D+L)\mathbf{u} = \mathbf{b} - U\mathbf{u} \\ &\iff \quad \mathbf{u} = \mathbf{u} + (D+L)^{-1}(\mathbf{b} - A\mathbf{u}) \end{aligned}$$

- solving $(D+L)\mathbf{z} = \mathbf{c}$ is $O(N)$ (for our A)

Definition

a Gauss-Seidel (GS) iteration applies a fast preconditioner:

$$\mathbf{v}_{k+1} = \mathbf{v}_k + (D+L)^{-1}(\mathbf{b} - A\mathbf{v}_k)$$

example of Gauss-Seidel iteration

- matrix-splitting view obscures the simplicity of Gauss-Seidel
- *example*: consider the linear system $A\mathbf{u} = \mathbf{b}$ with

$$A = \begin{bmatrix} 2 & -1 & & & & \\ -1 & 2 & -1 & & & \\ & -1 & 2 & -1 & & \\ & & \ddots & \ddots & \ddots & \\ & & & -1 & 2 & -1 \\ & & & & -1 & 2 \end{bmatrix}, \qquad \mathbf{b} = \begin{bmatrix} b_1 \\ b_2 \\ b_3 \\ \vdots \\ b_N \end{bmatrix}$$

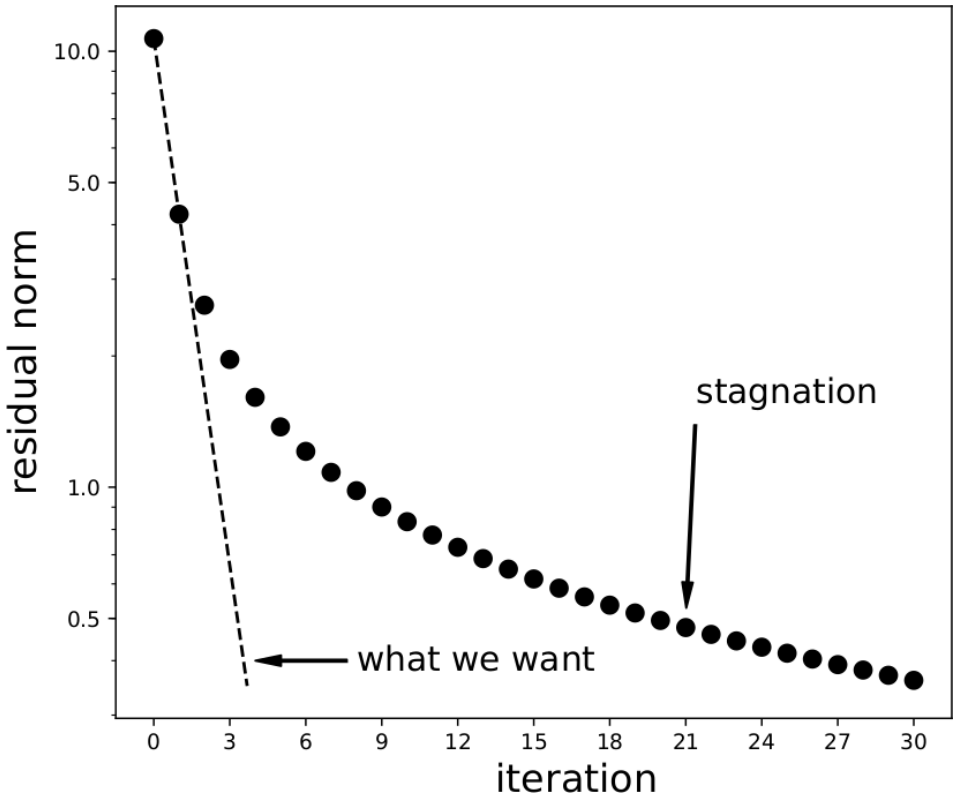
  - in this case, Gauss-Seidel iteration computes

$$v_j^{[k+1]} = \frac{b_j}{2} + \frac{1}{2}\left(v_{j-1}^{[k+1]} + v_{j+1}^{[k]}\right)$$

  - this is a relaxation method . . . update v_j using average of neighbors
  - one can prove this method converges
  - this example is relevant because $A \sim -\nabla^2$ in 1D

Jacobi and Gauss-Seidel iterations are bad *as solvers*

- for Poisson problem linear system $A\mathbf{u} = \mathbf{b}$, one can prove Gauss-Seidel and Jacobi converge
- but, after initial progress, residual norms decrease agonizingly slowly on fine grids
- an iteration $\mathbf{v}_{k+1} = \phi(\mathbf{v}_k)$ *stagnates* or *stalls* if the ratio of successive residual norms $\{\|\mathbf{r}(\mathbf{v}_{k+1})\|/\|\mathbf{r}(\mathbf{v}_k)\|\}$ goes to one



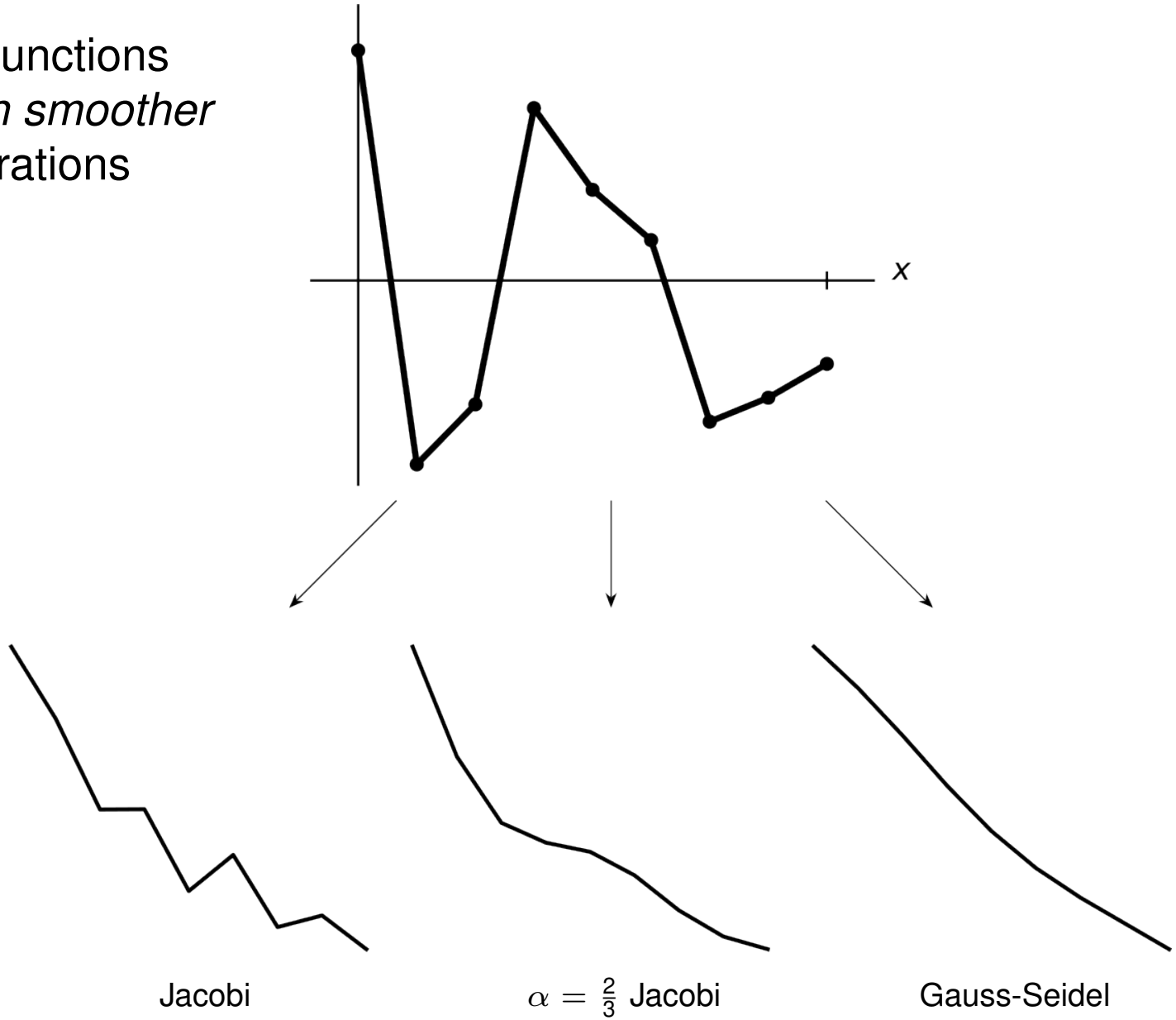
- Achi Brandt, an inventor of multigrid:

  Stalling numerical processes must be wrong. Whenever the computer grinds very hard for small or slow effect, there must be a better way to achieve the same goal.



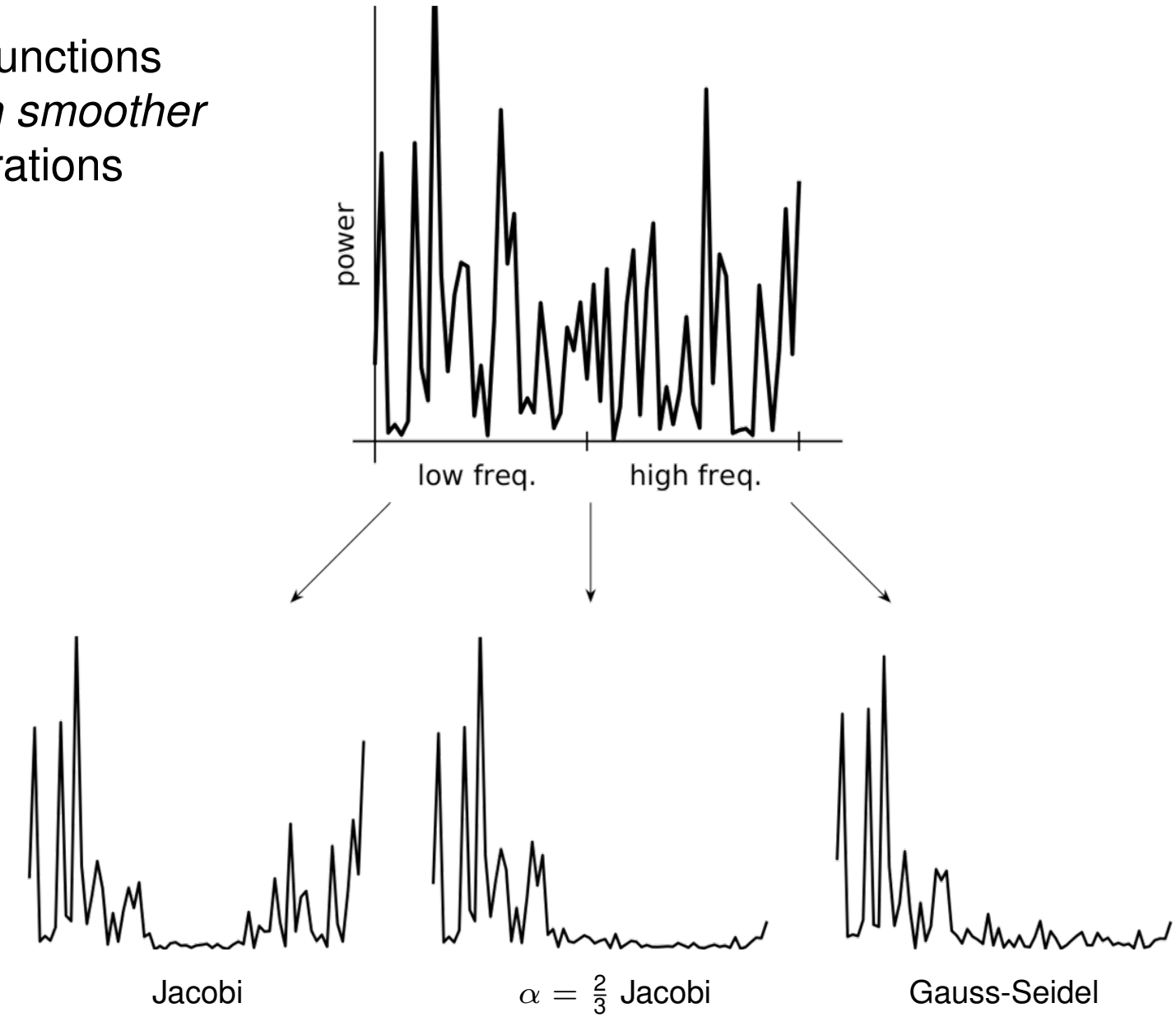
Jacobi and Gauss-Seidel iterations are good *as smoothers*

- observation: functions become *much smoother* after a few iterations



Jacobi and Gauss-Seidel iterations are good *as smoothers*

- observation: functions become *much smoother* after a few iterations

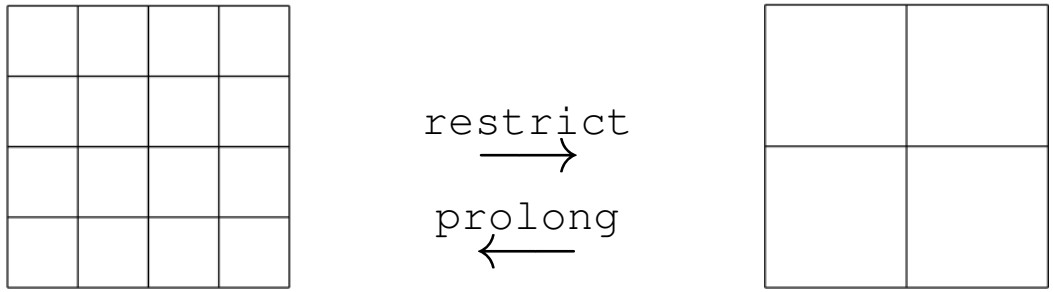


outline

1. examples: two elliptic PDE problems
2. simple iterations
3. 2-grid method, and the coarse-grid correction
4. V-cycle multigrid
5. nonlinear multigrid

grid transfers

- multiple grid resolutions will allow us to exploit smoothers to generate fast solutions, but we need grid transfer operators between the different grids



- for a function **v** defined on a finer grid, define its restriction $R\mathbf{v}$ to a coarser grid to be its value on the coarse grid points (injection), or restrict by averaging onto the coarser grid (*full weighting*)
- for a function **w** defined on the coarser grid, define its prolongation $P\mathbf{w}$ to a finer grid by (linear) *interpolation*
- R, P are linear operators
  - R, P are rectangular (non-square) matrices
  - $RP = I$, or approximately so

2-grid method

- put these ideas together!
- the 2-grid method approximately solves the PDE on the finer grid:

```
def twogrid(v,pre=2,post=2):
    for k in range(pre):
        v = smooth(A,b,v)
    rc = restrict(b - A*v)
    ec = solve(Ac,rc)              # solve error equation
    v = v + prolong(ec)
    for k in range(post):
        v = smooth(A,b,v)
    return v
```

- where

```
A,b   = discretize(m,m)          # fine grid
Ac,_ = discretize(m/2,m/2)       # coarse grid
```

- `smooth()` does one Jacobi or GS iteration
- `solve()` might be Gaussian elimination, etc., for $A^c\mathbf{e}^c = \mathbf{r}^c$

the error equation

- what do I mean by the “error equation”?
- for the linear system $A\mathbf{u} = \mathbf{b}$, consider some $\mathbf{v}$ which is *not* a solution

Definition

for any vector $\mathbf{v}$, the error equation corresponding to the linear system $A\mathbf{u} = \mathbf{b}$ is the equation

$$A\mathbf{e} = \mathbf{r}(\mathbf{v})$$

- here’s the logic:

$$\begin{aligned} \mathbf{e} &= \mathbf{u} - \mathbf{v} && \textit{definition of the error} \\ A\mathbf{e} &= A\mathbf{u} - A\mathbf{v} && \textit{multiply by } A \\ A\mathbf{e} &= \mathbf{b} - A\mathbf{v} = \mathbf{r}(\mathbf{v}) && \textit{error equation} \end{aligned}$$

the coarse grid correction

- the essential 3 lines in `twogrid()` form a coarse-grid correction:

```
rc = restrict(b - A*v)      # restrict the residual
ec = solve(Ac,rc)           # coarse-grid solve
v = v + prolong(ec)         # add back as correction
```

- this is a kind of simple iteration: $\mathbf{v} \leftarrow \mathbf{v} + P(A^c)^{-1}R(\mathbf{b} - A\mathbf{v})$
- define the *coarse-grid correction matrix*:

$$B^c = P(A^c)^{-1}R$$

- so `twogrid()` mixes two flavors of simple iteration:

$\mathbf{v} \leftarrow \mathbf{v} + M^{-1}(\mathbf{b} - A\mathbf{v})$ *the smoother*

$\mathbf{v} \leftarrow \mathbf{v} + B^c(\mathbf{b} - A\mathbf{v})$ *the coarse-grid correction*

- Q: is $B^c \approx A^{-1}$?

the coarse grid correction

- the essential 3 lines in `twogrid()` form a coarse-grid correction:

```
rc = restrict(b - A*v)      # restrict the residual
ec = solve(Ac,rc)           # coarse-grid solve
v = v + prolong(ec)         # add back as correction
```

- this is a kind of simple iteration: $\mathbf{v} \leftarrow \mathbf{v} + P(A^c)^{-1}R(\mathbf{b} - A\mathbf{v})$
- define the *coarse-grid correction matrix*:

$$B^c = P(A^c)^{-1}R$$

- so `twogrid()` mixes two flavors of simple iteration:

$\mathbf{v} \leftarrow \mathbf{v} + M^{-1}(\mathbf{b} - A\mathbf{v})$ *the smoother*

$\mathbf{v} \leftarrow \mathbf{v} + B^c(\mathbf{b} - A\mathbf{v})$ *the coarse-grid correction*

- Q: is $B^c \approx A^{-1}$? A: yes, but only for smooth inputs

2-grid method: the effect on error

- look at `twogrid()` again:

```
def twogrid(v,pre=2,post=2):
    for k in range(pre):
        v = smooth(A,b,v)
    rc = restrict(b - A*v)
    ec = solve(Ac,rc)
    v = v + prolong(ec)
    for k in range(post):
        v = smooth(A,b,v)
    return v
```

- a fairly-quick calculation shows that `twogrid()` applies a linear operator, which is close to the zero operator, to $\mathbf{e} = \mathbf{u} - \mathbf{v}$:

$$\mathbf{e} \leftarrow (I - M^{-1}A)^{\text{post}}(I - B^c A)(I - M^{-1}A)^{\text{pre}}\mathbf{e}$$

- Q: how should we solve the coarse-grid problem $A^c\mathbf{e}^c = \mathbf{r}^c$?

2-grid method: the effect on error

- look at `twogrid()` again:

```
def twogrid(v,pre=2,post=2):
    for k in range(pre):
        v = smooth(A,b,v)
    rc = restrict(b - A*v)
    ec = solve(Ac,rc)
    v = v + prolong(ec)
    for k in range(post):
        v = smooth(A,b,v)
    return v
```

- a fairly-quick calculation shows that `twogrid()` applies a linear operator, which is close to the zero operator, to $\mathbf{e} = \mathbf{u} - \mathbf{v}$:

$$\mathbf{e} \leftarrow (I - M^{-1}A)^{\text{post}}(I - B^c A)(I - M^{-1}A)^{\text{pre}}\mathbf{e}$$

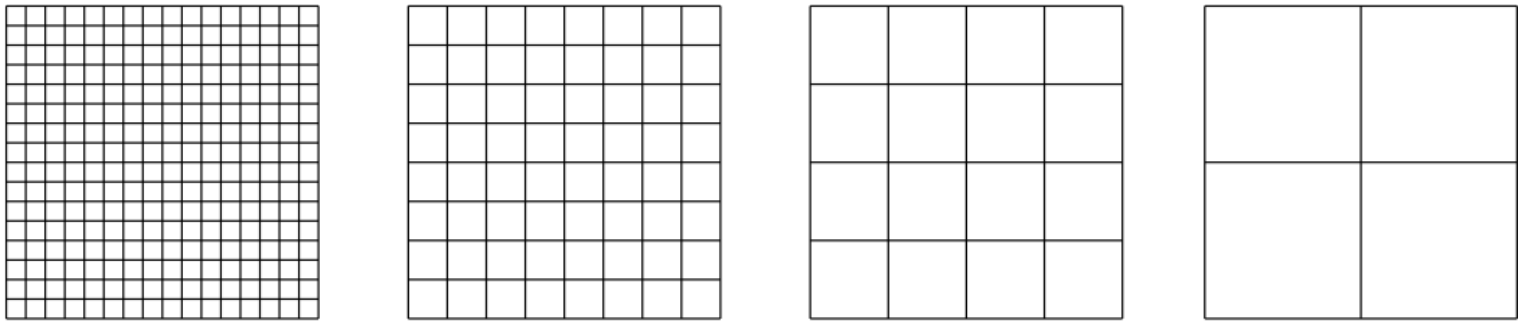
- Q: how should we solve the coarse-grid problem $A^c\mathbf{e}^c = \mathbf{r}^c$?

outline

1. examples: two elliptic PDE problems
2. simple iterations
3. 2-grid method, and the coarse-grid correction
4. V-cycle multigrid
5. nonlinear multigrid

hierarchy of grids

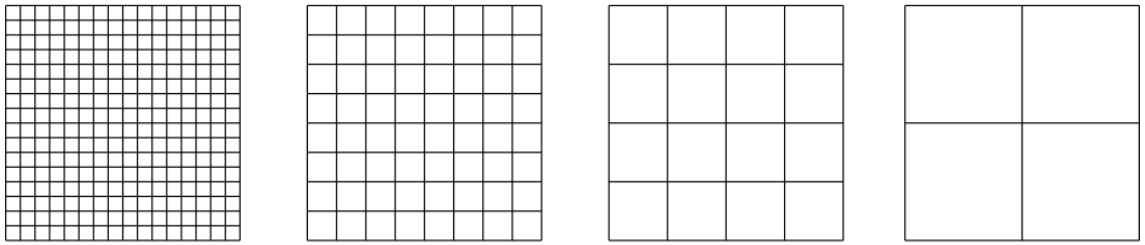
- A: if we can have two levels of grids, we can have many!
- when faced with a coarse-grid solve, just do another 2-grid . . . and keep going down to some really easy and cheap coarse grid



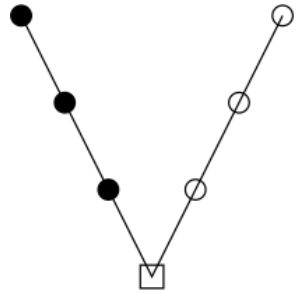
- restrictions R and prolongations P are needed in this grid hierarchy

recursive V-cycle

```
def vcycle(b,v,lev,pre=2,post=2):
    A,_ = discretize(lev)
    if lev == 0:
        return solve(A,b)      # the buck stops here
    for k in range(pre):
        v = smooth(A,b,v)
    rc = restrict(b - A*v)
    ec = vcycle(r,0,lev-1)     # descend a grid level
    v = v + prolong(ec)
    for k in range(post):
        v = smooth(A,b,v)
    return v
```



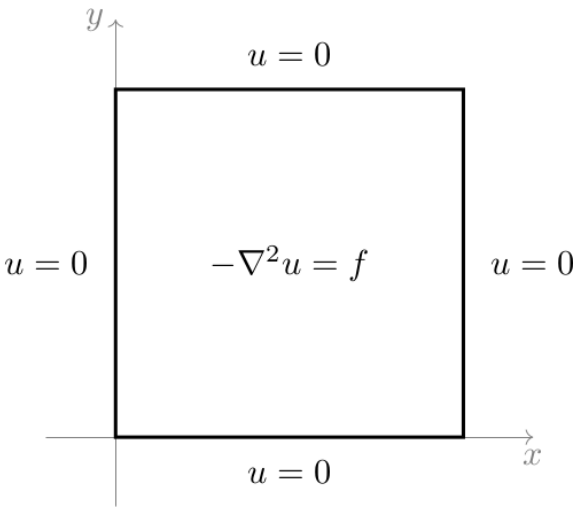
$\Omega^{(3)}$ $\Omega^{(2)}$ $\Omega^{(1)}$ $\Omega^{(0)}$

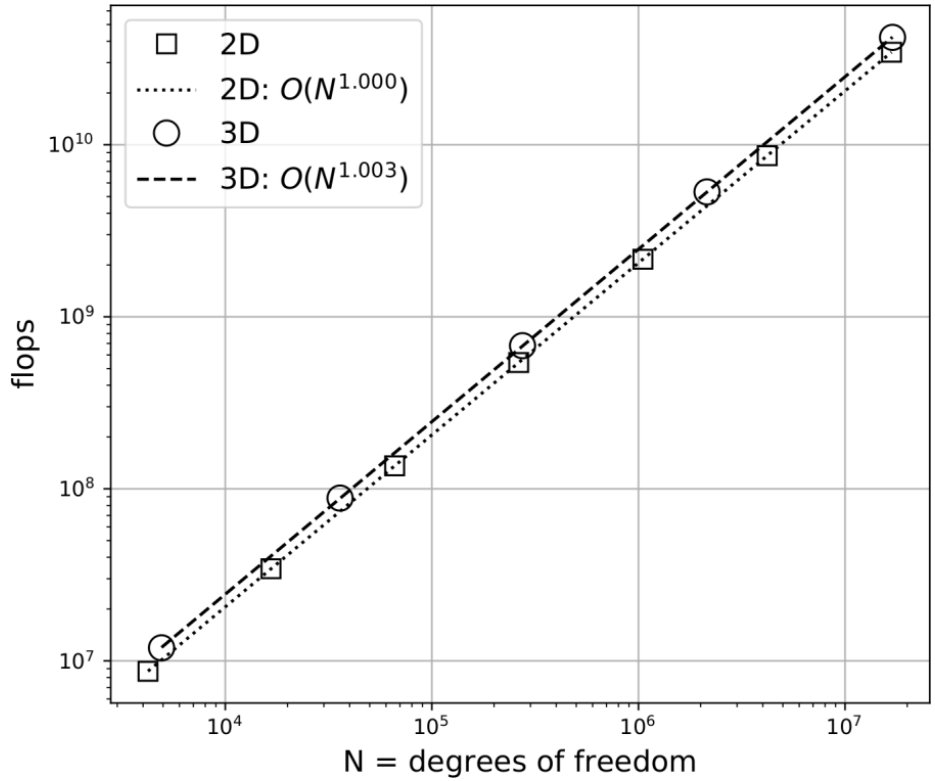


how well does it work?

- V-cycles work absurdly well on the Poisson problem $-\nabla^2 u = f$

- scaling out to $m = 4097$ ($N = 1.6 \times 10^7$):

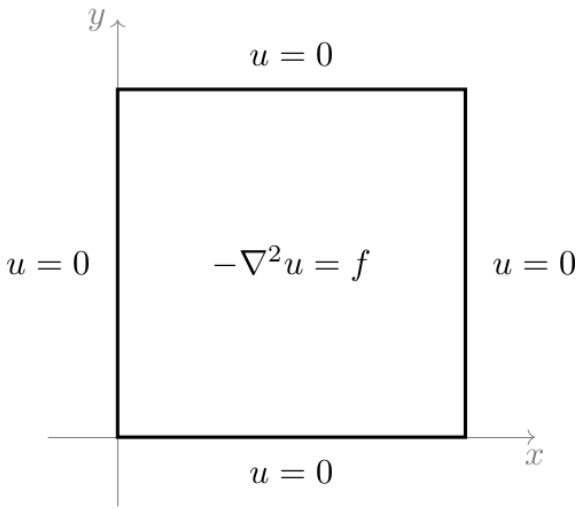


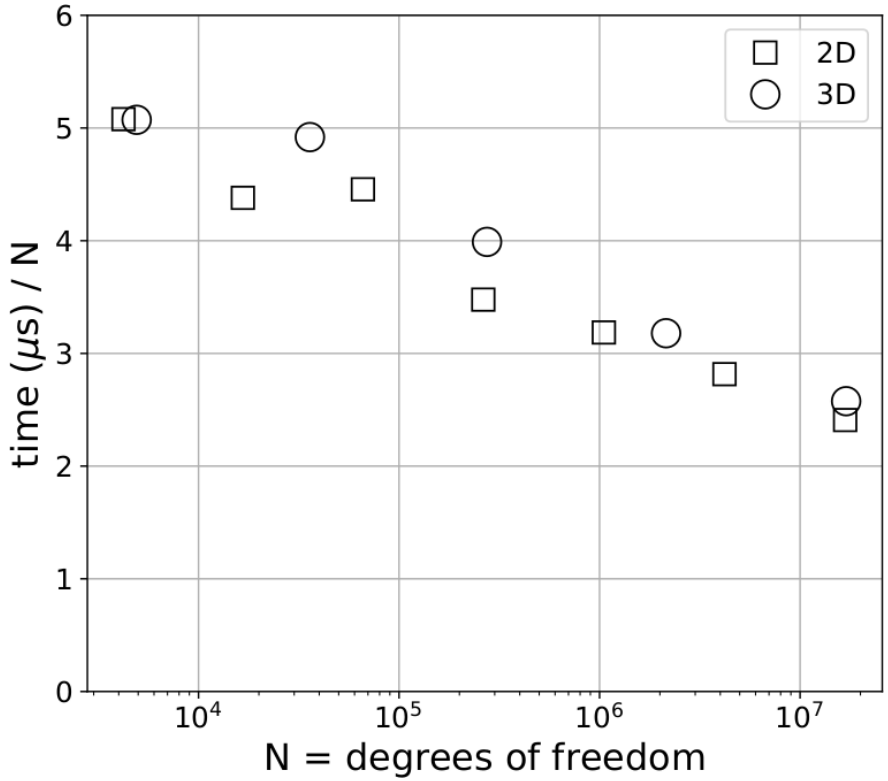


how well does it work?

- V-cycles work absurdly well on the Poisson problem $-\nabla^2 u = f$

- scaling out to $m = 4097$ ($N = 1.6 \times 10^7$):





on multigrid costs: single V-cycle

- let us compute the work (flops) of applying a single V-cycle
  - note: multiple V-cycles are generally needed to solve the problem

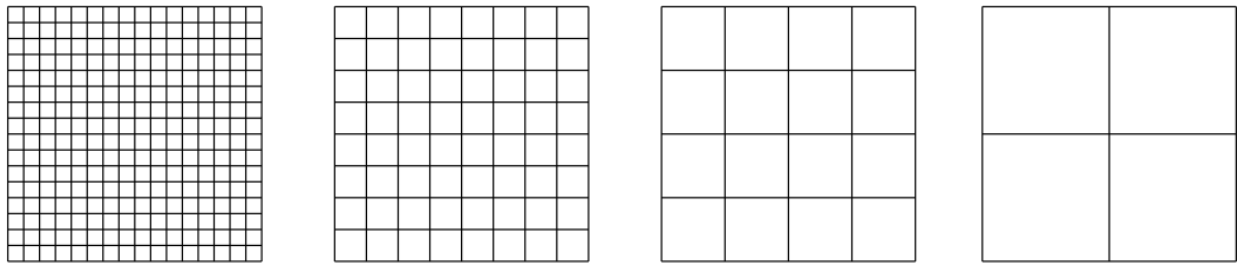
Definitions

$$|\Omega^{(k)}| = \text{(number of grid points (unknowns) on grid } \Omega^{(k)}\text{)}$$

$$W_k^{k-1} = \begin{pmatrix} \text{smoother work done on grid } \Omega^{(k)}\text{, plus cost of} \\ \text{restriction/prolongation to/from next-coarser grid } \Omega^{(k-1)} \end{pmatrix}$$

$$W_0 = \text{(solver work done on the coarsest level)}$$

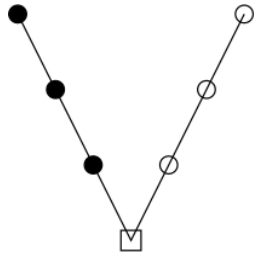
- $K = 3$ case:



on multigrid costs: single V-cycle

- total cost of a single V-cycle:

$$\overline{W} = W_K^{K-1} + W_{K-1}^{K-2} + \cdots + W_1^0 + W_0$$



- for 2D grids, each coarse grid is 4 times smaller:

$$|\Omega^{(k-1)}| \approx \tfrac{1}{4}|\Omega^{(k)}|$$

- since smoothers and restriction/prolongation are $O(1)$ per grid point:

$$W_k^{k-1} \leq C|\Omega^{(k)}|$$

  - for some C independent of k

- since $N = |\Omega^{(K)}|$ is the number of points in the finest grid,

$$\begin{aligned}
\overline{W} &\leq C|\Omega^{(K)}| + C|\Omega^{(K-1)}| + \cdots + C|\Omega^{(1)}| + W_0 \\
&\approx CN\left(1 + \tfrac{1}{4} + \cdots + \tfrac{1}{4^{K-1}}\right) + W_0 \\
&\approx CN\tfrac{1}{1-(1/4)} = \tfrac{4}{3}CN
\end{aligned}$$

optimal

summary so far

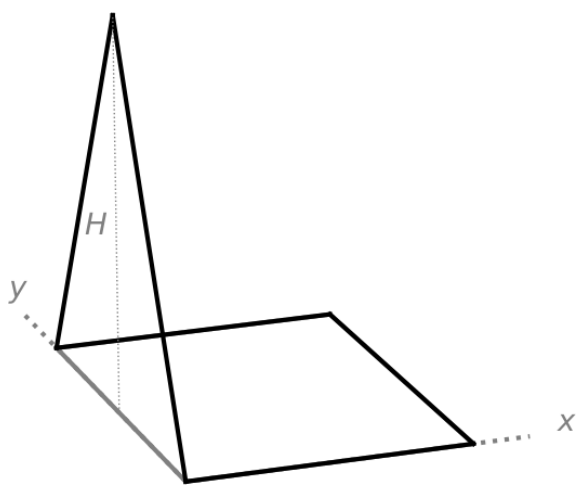
- multigrid combines 3 concepts:
  1. classical iterations, such as Jacobi and GS, are cheap smoothers of the residual and the error
  2. a coarse-grid correction does a good job of updating the fine-grid solution, if acting on a smooth residual
  3. the coarse-grid correction is cheap because restriction and prolongation are cheap

- but the Poisson problem is too easy!

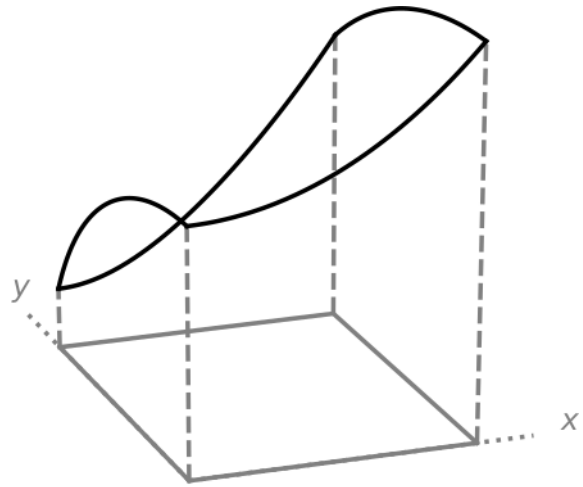
outline

1. examples: two elliptic PDE problems
2. simple iterations
3. 2-grid method, and the coarse-grid correction
4. V-cycle multigrid
5. nonlinear multigrid

minimal surfaces, a nonlinear problem



tent



catenoid

minimal surface problem

for given boundary function (wire frame) $g(x, y)$, find $u(x, y)$ so that

$$-\nabla \cdot \left(\frac{\nabla u}{\sqrt{1 + |\nabla u|^2}} \right) = 0 \text{ on } \Omega, \qquad u\big|_{\partial\Omega} = g$$

discretization gets you . . . more general equations

- at each point (x_i, y_j) on the target (finest) grid $\Omega^{(K)}$ we discretize to get an FD approximation of the PDE:

$$f_{i,j}(u_{i-1,j},\ u_{i+1,j},\ u_{i,j-1},\ u_{i,j+1},\ u_{i,j}) = 0$$

  - unknowns are globally-ordered into a vector $\mathbf{u} \in \mathbb{R}^N$: $u_\ell = u_{i,j}$
  - also globally-order the equations: $f_\ell(\mathbf{u}) = f_{i,j}(\mathbf{u})$

nonlinear discretization principle

enforcing the PDE at grid point (x_i, y_j) gives one scalar equation $f_{i,j}(\mathbf{u}) = 0$

- get a nonlinear system of N scalar equations in N scalar unknowns:

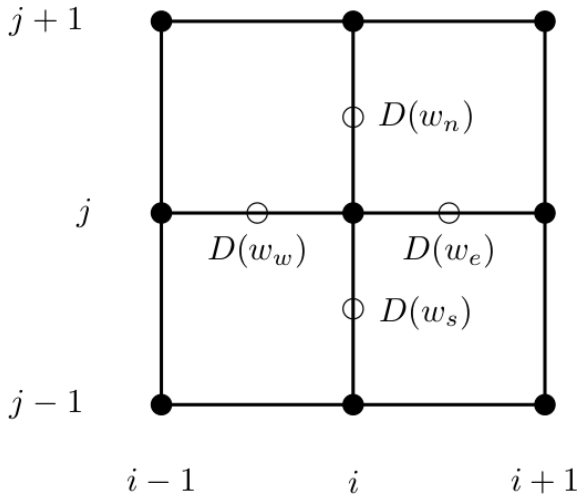
$$\mathbf{F}(\mathbf{u}) = \mathbf{0}$$

- $\mathbf{F}$ is a sparse function, as each f_ℓ depends on only $O(1)$ entries of $\mathbf{u}$

details: 9-point stencil with staggered diffusivity

- how do you discretize $\nabla \cdot \left(\dfrac{\nabla u}{\sqrt{1+|\nabla u|^2}} \right)$?
- generalize to $\nabla \cdot (D(w)\nabla u)$ where

$$D(w) = (1+w)^{-1/2}$$
$$w = |\nabla u|^2$$



- centered FD, using *staggered* values of $D(w)$:

$$\nabla \cdot (D(w)\nabla u) \approx \frac{D(w_e)(u_{i+1,j} - u_{i,j}) - D(w_w)(u_{i,j} - u_{i-1,j})}{h_x^2} + \frac{D(w_n)(u_{i,j+1} - u_{i,j}) - D(w_s)(u_{i,j} - u_{i,j-1})}{h_y^2}$$

$$w_e = \left[|\nabla u|^2 \right]_{i+\frac{1}{2},j} \approx \left(\frac{u_{i+1,j} - u_{i,j}}{h_x} \right)^2 + \left(\frac{u_{i,j+1} + u_{i+1,j+1} - u_{i,j} - u_{i+1,j}}{4h_y} \right)^2$$

Newton's method

- Q: how do we solve our nonlinear system $\mathbf{F}(\mathbf{u}) = \mathbf{0}$?
  A: Newton's method:

$$J(\mathbf{u}^p)\,\mathbf{s} = -\mathbf{F}(\mathbf{u}^p)$$
$$\mathbf{u}^{p+1} = \mathbf{u}^p + \mathbf{s}$$

  using the Jacobian matrix $J(\mathbf{v})_{r,s} = \left[\frac{\partial f_r(\mathbf{v})}{\partial v_s} \right]$

- Q: how do you calculate the Jacobian?
  A: compute derivatives on paper, and write code to evaluate it
  A: finite differencing again: $J(\mathbf{v})_{r,s} \approx \frac{f_r(\mathbf{v} + \epsilon \mathbf{1}_s) - f_r(\mathbf{v})}{\epsilon}$
  A: symbolically, and using powerful libraries

Newton's method and Jacobian management

- Q: how do we solve our nonlinear system $\mathbf{F}(\mathbf{u}) = \mathbf{0}$?
  A: Newton's method:

$$J(\mathbf{u}^p)\,\mathbf{s} = -\mathbf{F}(\mathbf{u}^p)$$
$$\mathbf{u}^{p+1} = \mathbf{u}^p + \mathbf{s}$$

  using the Jacobian matrix $J(\mathbf{v})_{r,s} = \left[\dfrac{\partial f_r(\mathbf{v})}{\partial v_s} \right]$

- Q: how do you calculate the Jacobian?

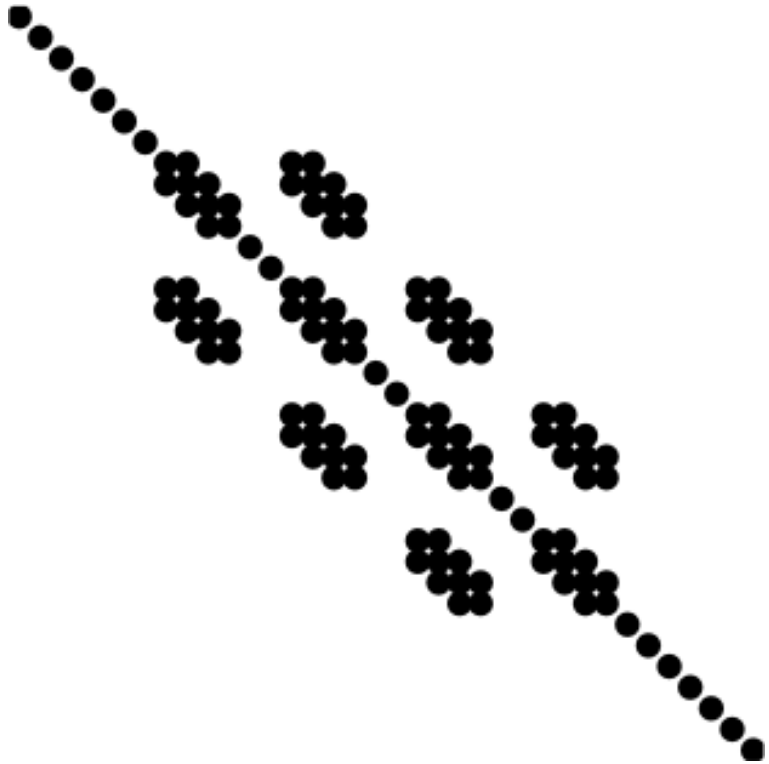
  A: compute derivatives on paper, and write code to evaluate it

  A: finite differencing again: $J(\mathbf{v})_{r,s} \approx \dfrac{f_r(\mathbf{v} + \epsilon \mathbf{1}_s) - f_r(\mathbf{v})}{\epsilon}$

  A: symbolically, and using powerful libraries

details: 9-point stencil Jacobian sparsity

- on a 6×6 grid, the Jacobian $J(\mathbf{v})$ has this sparsity pattern:



Newton-multigrid

- Q: how do you solve each linear system in the Newton iteration?

  A: solve $J(\mathbf{u}^p)\,\mathbf{s} = -\mathbf{F}(\mathbf{u}^p)$ using multigrid:

```
def newtonmultigrid(v,cycles=1,levs):
    for p in ...:
        b = -F(v)
        s = 0
        for _ in range(cycles):
            s = vcycle(b,s,levs)
        v = v + s
    return v
```

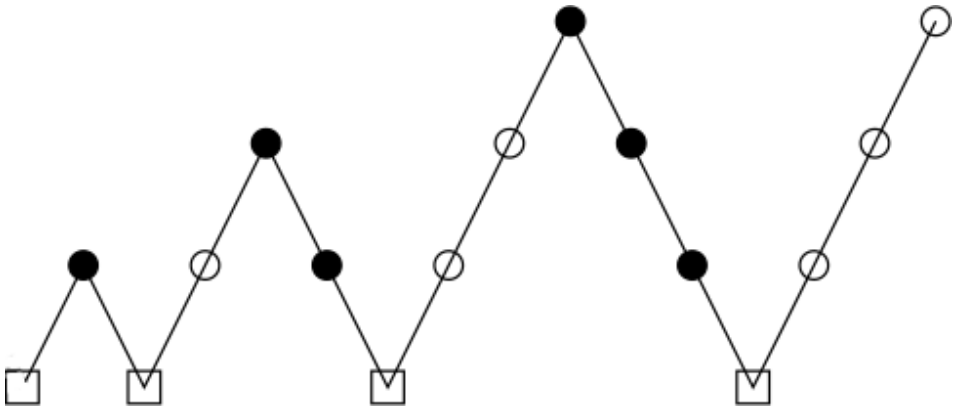
nonlinear multigrid F-cycle solvers

- Q: how do you find a good initial iterate $\mathbf{u}^0$ for the Newton iteration?

  A: by solving the problem on a coarser grid, and prolonging

- this strategy is grid-sequencing
  - justification: the domain of Newton convergence is larger on the smaller (= coarser) version of the PDE

- if you also solve at each level with Newton-multigrid, then this is a nonlinear F-cycle multigrid . . . the most powerful solver!



minimal surface PDE problem: **live** demo

- one open-source library which fully supports multigrid is PETSc
  - PETSc = portable, extensible toolkit for scientific computing `petsc.org`
  - it is inside the Python finite element library Firedrake `firedrakeproject.org`
- this demo comes from Chapter 7 of my book *PETSc for Partial Differential Equations* `github.com/bueler/p4pdes`

```
$ cd p4pdes/c/ch7/
$ make minimal

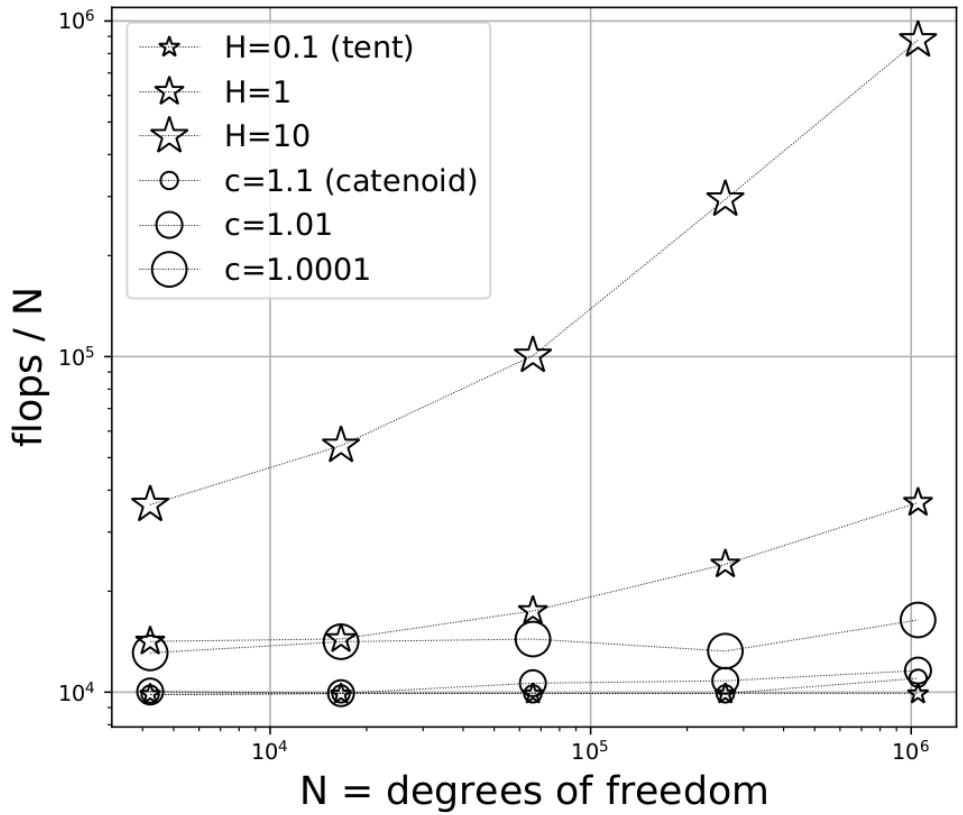
$ ./minimal -snes_fd_color -pc_type mg \
  -snes_grid_sequence 5 -snes_converged_reason

$ ./minimal -ms_problem tent -snes_fd_color -pc_type mg \
  -snes_grid_sequence 8 -snes_converged_reason \
  -snes_monitor_solution draw

$ mpiexec -n 6 ./minimal -ms_problem tent -snes_fd_color -pc_type mg \
  -snes_grid_sequence 9 -snes_converged_reason \
  -snes_view_solution vtk:u.vtr
$ paraview u.vtr
```

minimal surface PDE problem: the performance measurement

- amount of arithmetic (flops) per degree of freedom (N)?
- result: the solver is optimal on this nonlinear minimal surface equation
  - but less so if the tent door is high



summary

- a linear multigrid V-cycle combines concepts to build an optimal solver for linear elliptic PDEs:
  1. classical iterations are cheap smoothers
  2. coarse-grid corrections act accurately on a smooth residual
  3. restriction and prolongation are cheap
- for nonlinear elliptic PDEs:
  1. wrap a Newton iteration around linear V-cycles: Newton-multigrid
  2. add grid-sequencing to generate initial iterates on each finer grid
  3. result nonlinear multigrid F-cycle, an optimal solver in many cases
- other flavors of multigrid: ← different talks!
  - full-approximation scheme (avoids the linearization in Newton's method)
  - linear algebraic multigrid (avoids hierarchy of grids)

references for learning about these ideas

A. Brandt (1977). *Multi-level adaptive solutions to boundary-value problems*, Mathematics of Computation 31 (138), 333–390

- the origin of powerful multigrid, by its guru



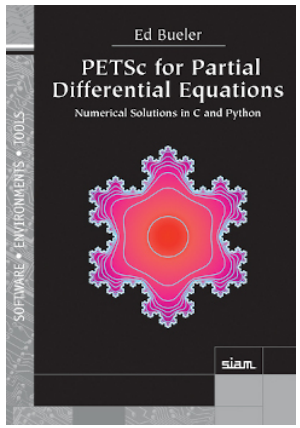
Achi Brandt

W. Briggs, V. E. Henson, & S. McCormick (2000). *A Multigrid Tutorial*, 2nd ed., SIAM Press, Philadelphia

- smaller, straightforward introduction

E. Bueler (2021). *PETSc for Partial Differential Equations*, SIAM Press, Philadelphia

- preconditioning ideas and diverse multigrid examples
- for open-source C and Python codes, follow QR ↘



H. Elman, D. Silvester, & A. Wathen (2014). *Finite Elements and Fast Iterative Solvers: With Applications to Incompressible Fluid Dynamics*, 2nd ed., Oxford U. Press

- multigrid in FE and fluids contexts

U. Trottenberg, C. Oosterlee, & A. Schuller (2001). *Multigrid*, Elsevier

- comprehensive view, and theory